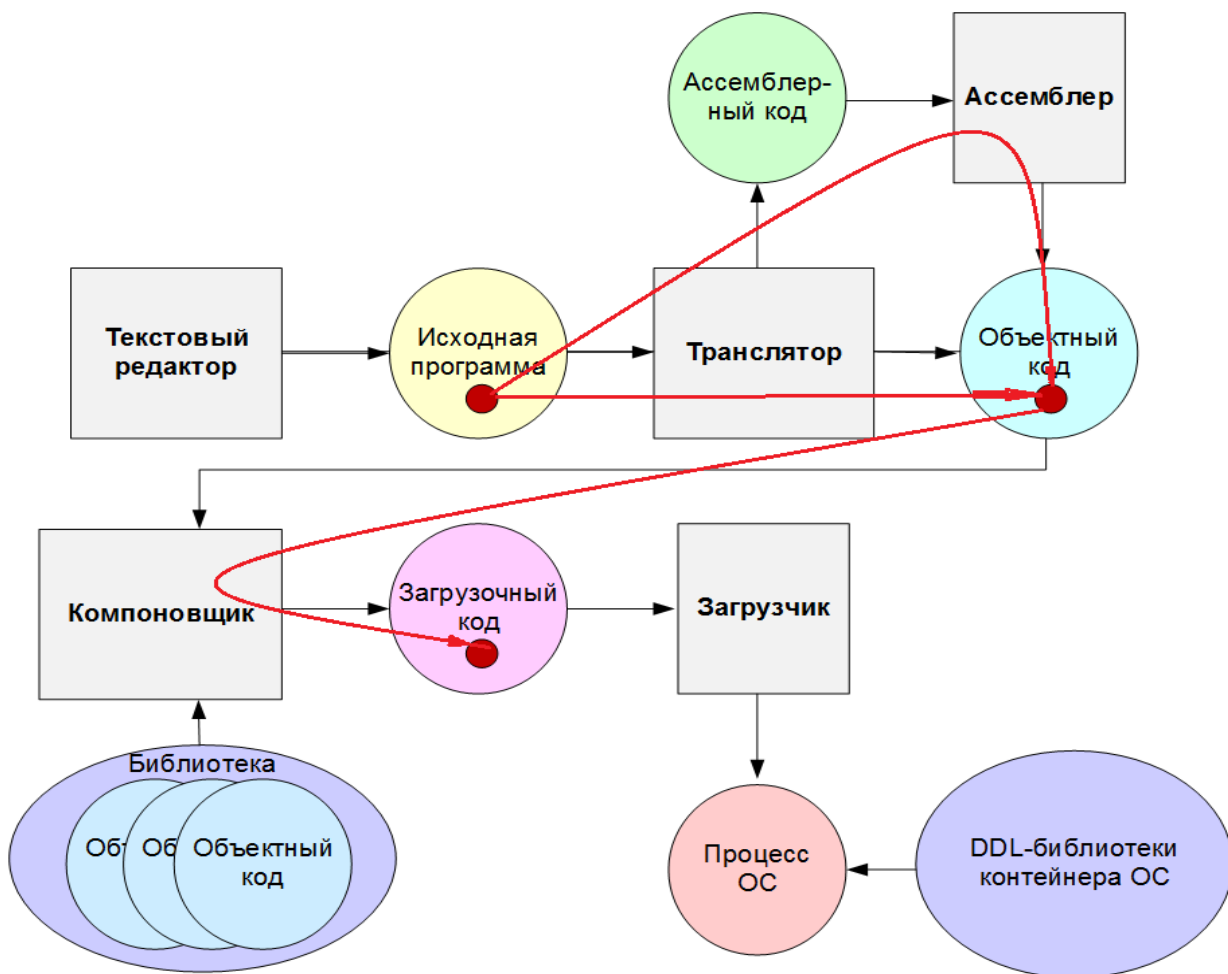


Генерация кода

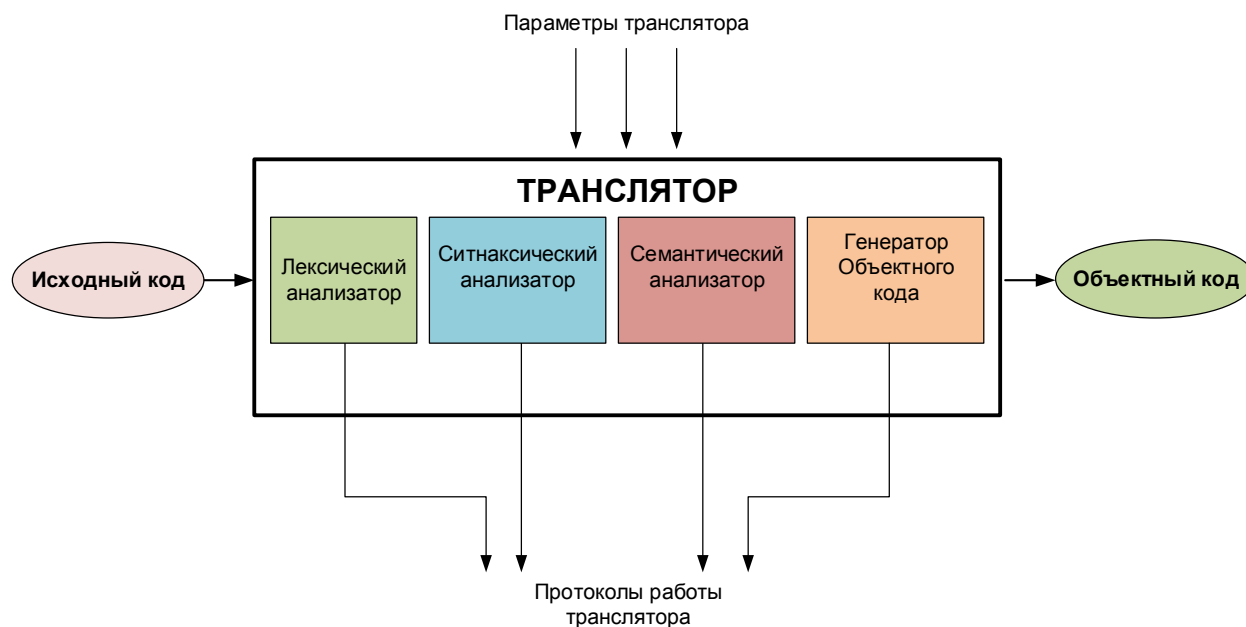
1. Система программирования

Структура системы программирования:

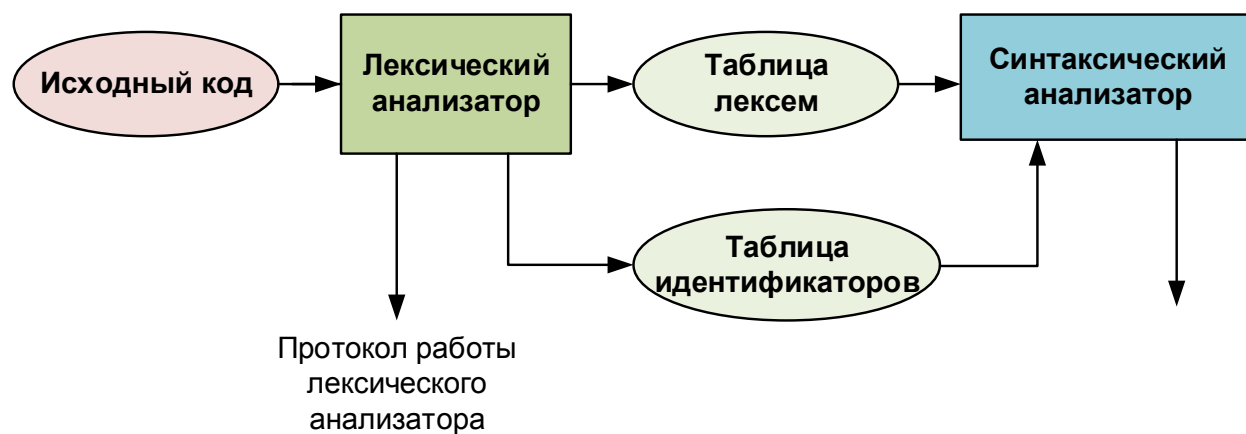


2. Транслятор: общая схема, фазы трансляции

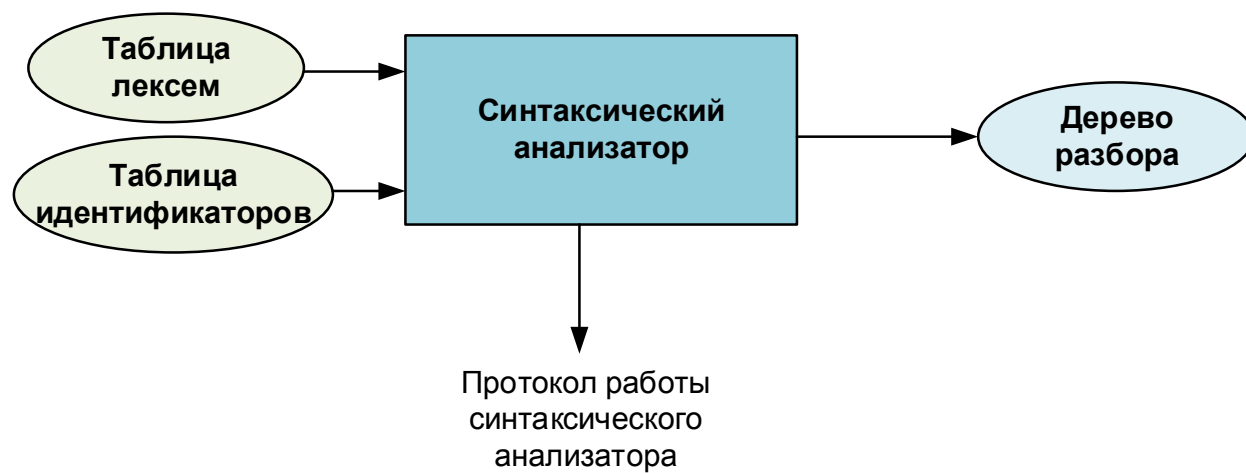
Общая схема транслятора:

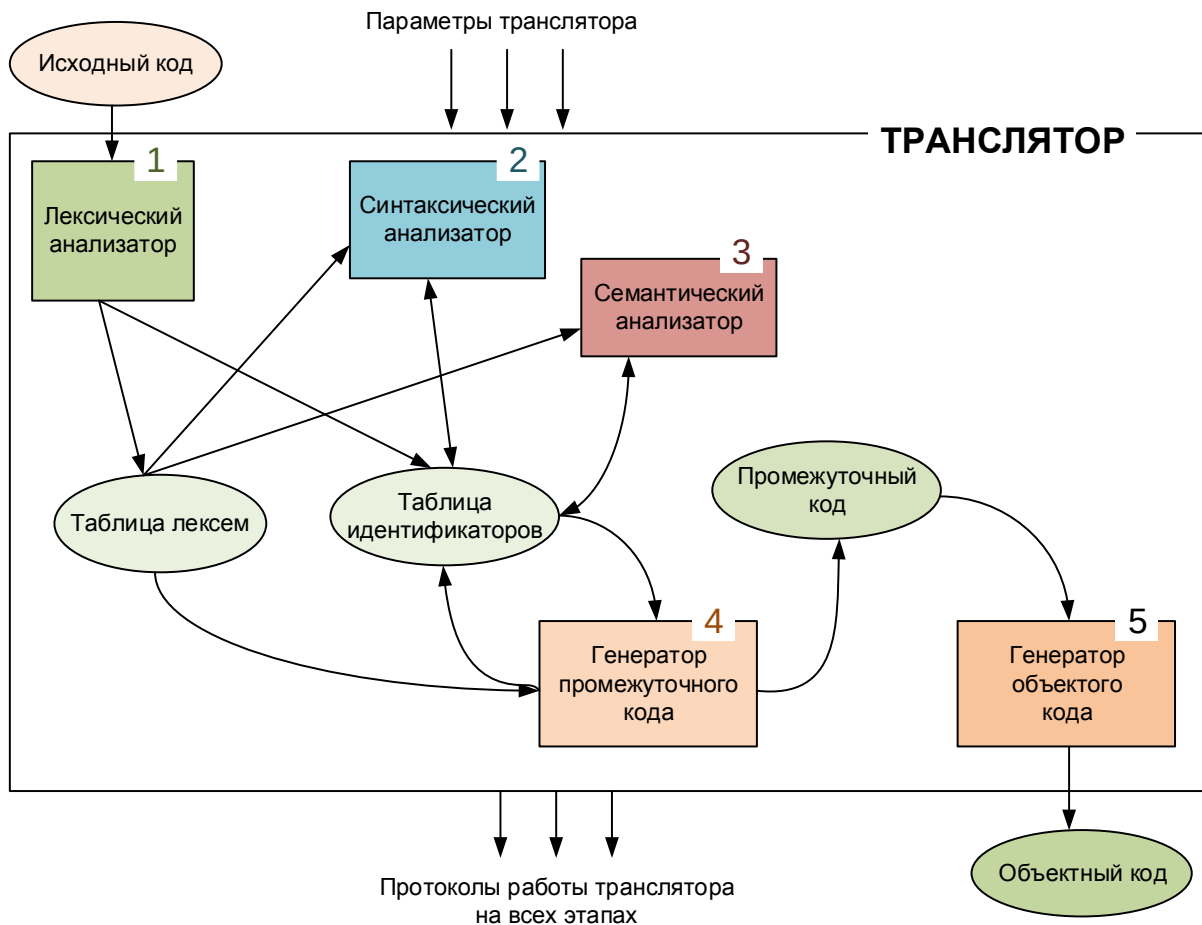


Фазы трансляции (лексический анализ):



Фазы трансляции (синтаксический анализ):





Основной задачей генератора кода является преобразование кода из промежуточного представления в эквивалентный код для целевой машины.

Входом генератора кода является промежуточное представление исходной программы.

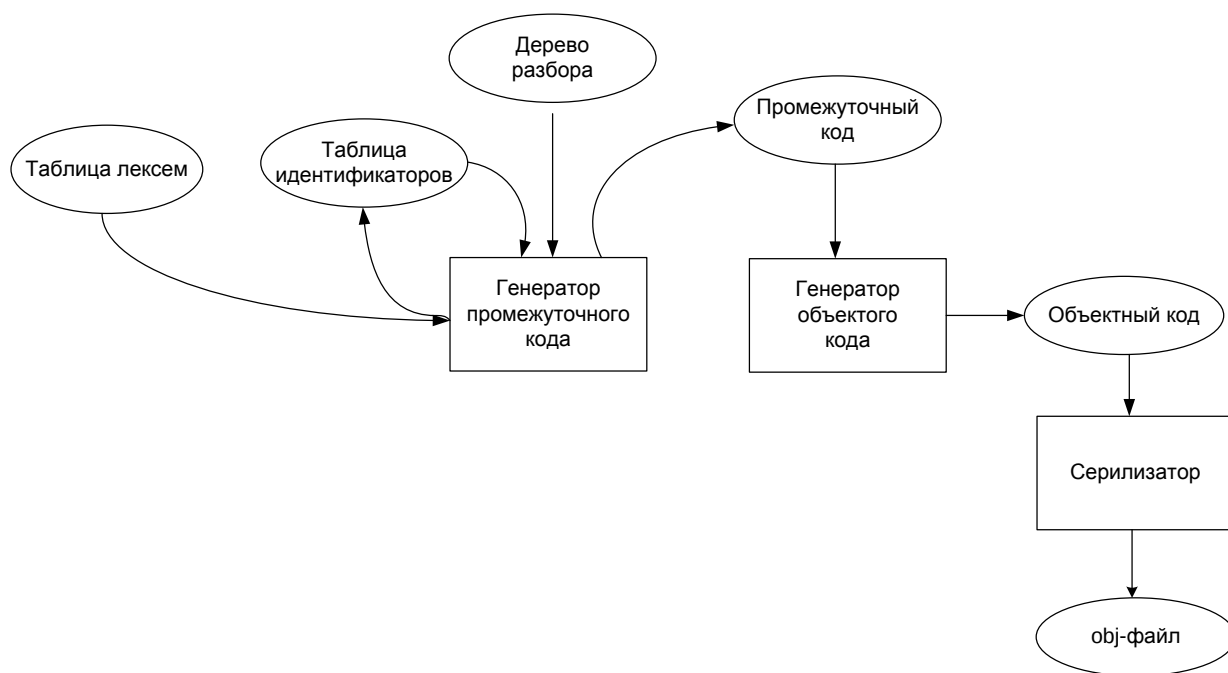
Выходом генератора кода является код для целевой машины, эквивалентный исходному коду.

Генерацию кода можно разделить на две части:

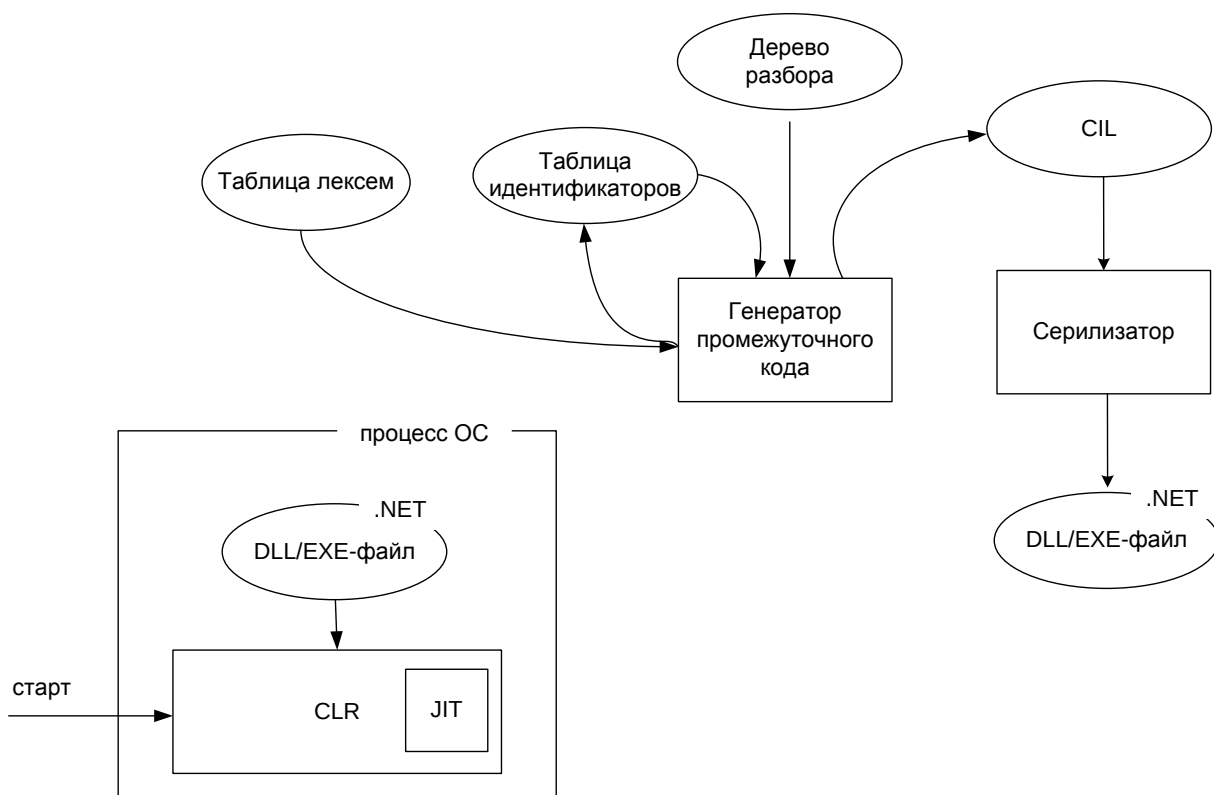
1. генерация промежуточного кода (не зависит от платформы);
2. генерация объектного кода на основе промежуточного (для конкретной платформы).

Процесс генерации кода может управляться параметрами.

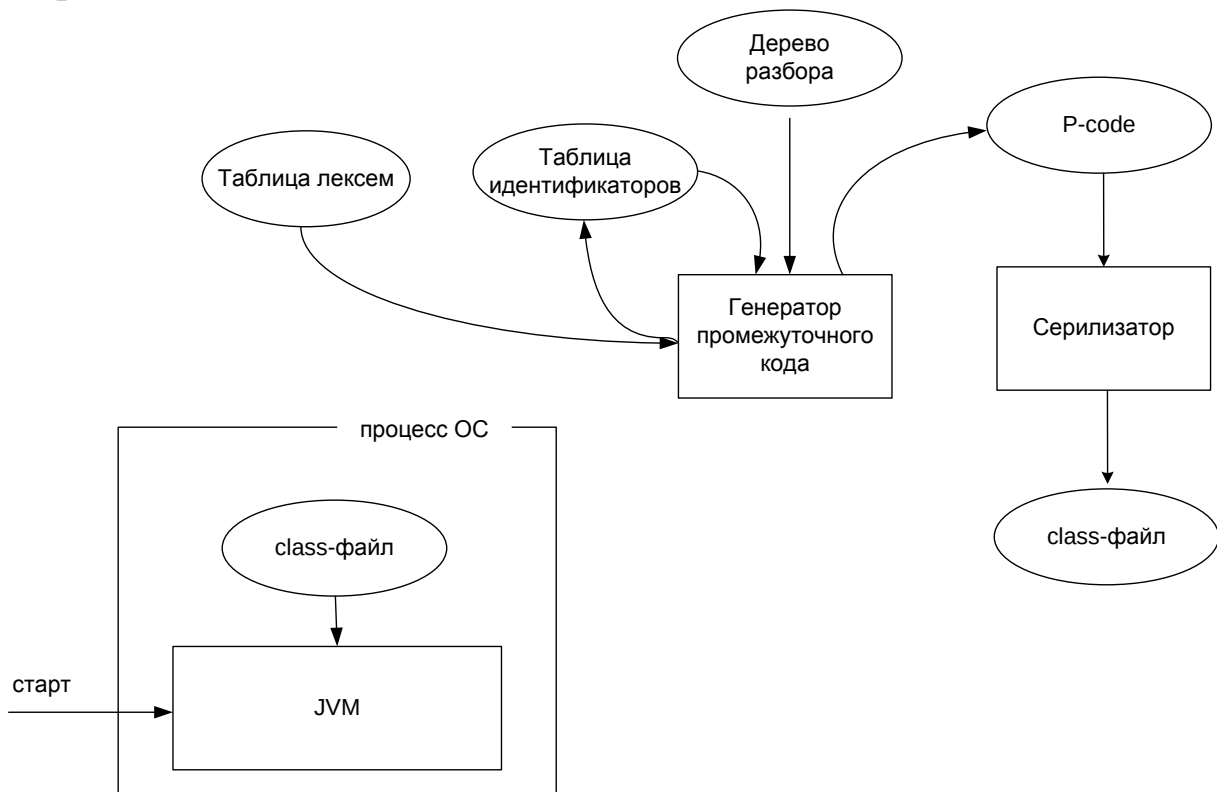
3. Генерация кода C/C++



4. Генерация кода языка .NET (C#, VB.NET)



5. Генерация кода языка Java

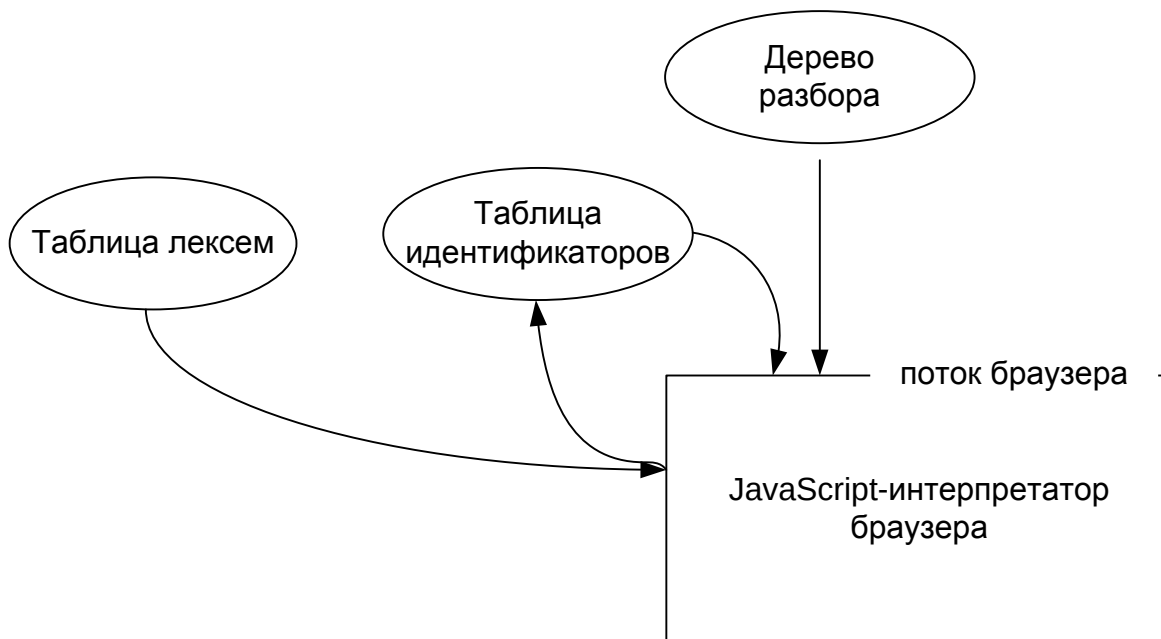


6. Сериализация: процесс преобразование структуры данных в последовательность битов.

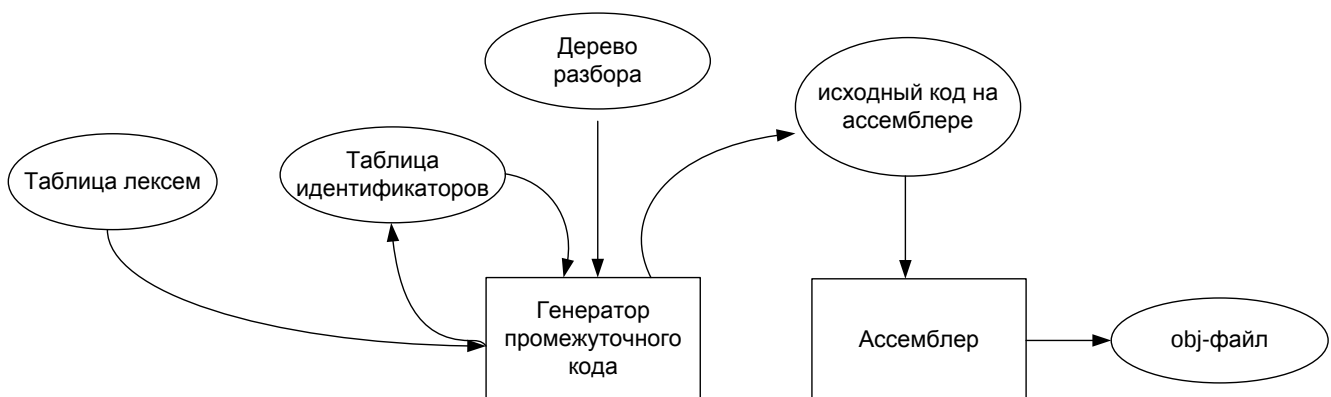
7. Десериализация: процесс преобразования последовательности битов в структуру данных.

8. Курсовой проект

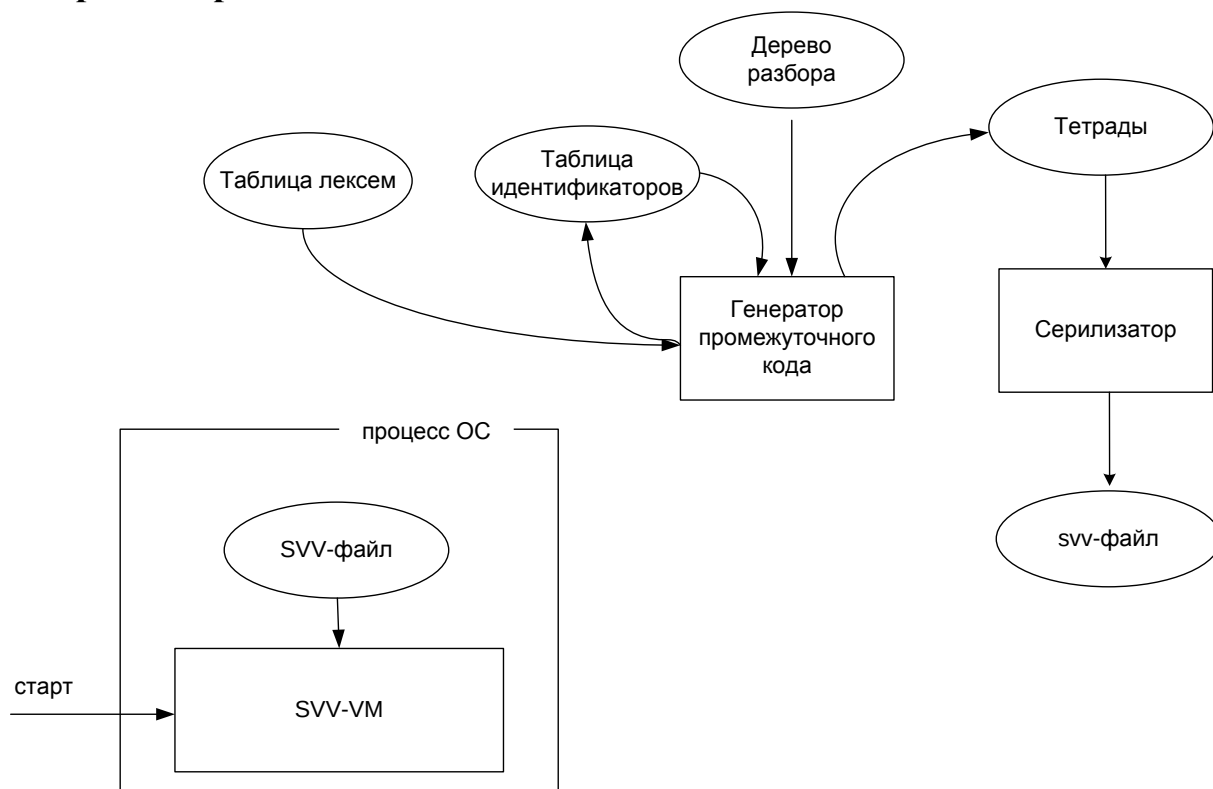
а. Интерпретация кода JavaScript



б. Генерация исходного ассемблерного кода



с. Принцип реализации SVV-2015



Генератор промежуточного кода упакует тетрады и сохранит в файл.

Загрузчик – десериализует, разместит в памяти и передаст управление.

Загрузка/десериализация



9. Генерация исходного ассемблерного кода

1) Объединение процедур, расположенных в разных модулях

Процедура — именованная, правильным образом оформленная группа команд, которая объявляется один раз и может многократно вызываться по имени в любом месте программы.

MASM:

EXTRN <имя> — объявление внешнего имени по отношению к данному модулю.

PROC и ENDP — начало, конец процедуры.

CALL <ИМЯ_ПРОЦЕДУРЫ> — команда вызова процедуры.

RET <ЧИСЛО> — команда возврата управления вызывающей программе,
где <число> — количество байт, удаляемых из стека при
возврате из процедуры (необязательный параметр).

Объединение процедур, расположенных в разных модулях

Каждый модуль должен сообщать транслятору, что некоторый объект (процедура, переменная) должен быть видимым вне этого модуля.

Транслятор также должен знать, что некоторые объекты определены вне данного модуля.

Все внешние ссылки в объединяемых модулях разрешаются на этапе **компоновки**.

Организация интерфейса с процедурой

Для передачи аргументов в языке ассемблера существуют следующие способы:

- через регистры;
- через общую область памяти;
- через стек;
- с помощью директив **extern** и **public**.

Передача аргументов через стек

Вызывающая процедура заносит в стек параметры и передает управление **вызываемой** процедуре. При передаче управления процедуре в вершину стека поверх параметров автоматически записывается 4 байта с адресом возврата в вызывающую программу.

Стек обслуживается тремя регистрами:

- ESI — указатель дна стека (начала сегмента стека);
- ESP — указатель вершины стека;
- EBP — указатель базы.

Регистры ESI и ESP указывают на дно и вершину стека соответственно.
 Регистр EBP используется для произвольного доступа к данным в стеке.

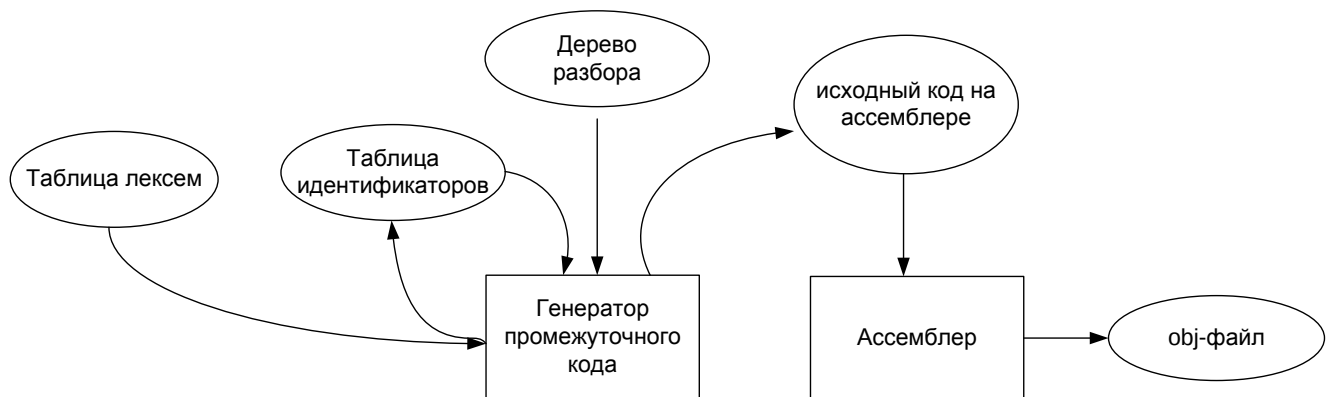
Пролог процедуры (настройка регистра EBP в процедуре)	<code>push ebp</code> // указатель базы сохраняется в стеке <code>mov ebp, esp</code> // настраивает EBP на вершину стека
Эпилог процедуры	Восстановление стека // выполняет действия, обеспечивающие // корректный возврат из процедуры

Восстановление стека:

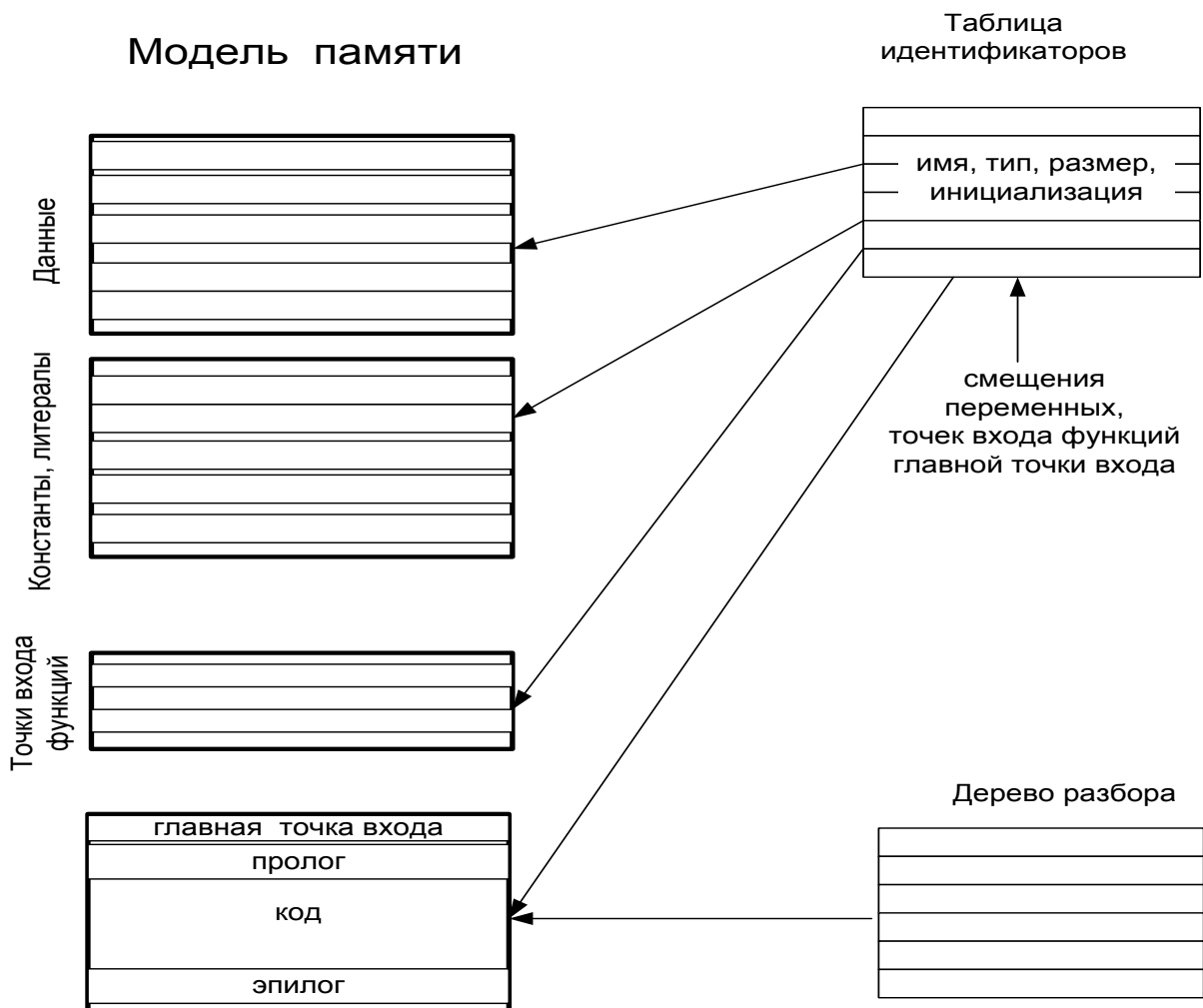
<i>cdecl</i>	<code>add esp, <длина_параметров_в_байтах></code>	// в вызывающем коде
<i>stdcall</i>	<code>ret <длина_параметров_в_байтах></code>	// в вызываемом коде

В программах, написанных на языке ассемблера, используется соглашение о вызовах `stdcall`.

2) Подход к генерации кода по дереву разбора с использованием стека.



3) Модель памяти SVV-2015 (статическая)



Прологом называют часть кода функции, который используется:

- для настройки стекового фрейма (сохранение оригинального содержимого регистра ESP, настройку нового и инициализацию указателя кадра EBP);
- для сохранения значений регистров (через которые в функцию переданы аргументы) в локальный стек для последующей работы с ними в коде функции;
- для сохранения (в стеке) значений регистров, которые будут использованы внутри подпрограммы, поскольку код должен заботиться о том, чтобы значения регистров процессора, до и после работы функции оставались неизменными;
- для резервирования места в стеке с целью хранения локальных переменных. Достигается это смещением указатель стека в сторону уменьшения адресов, то есть в сторону увеличения стека. После этого мы получаем часть памяти, которая доступна (через указатель фрейма) и не задействована. Это пространство может быть использовано кодом функции по своему усмотрению, чаще всего для хранения локальных переменных функции.

Эпилог – код, обеспечивающий восстановление стека, согласование с ОС.

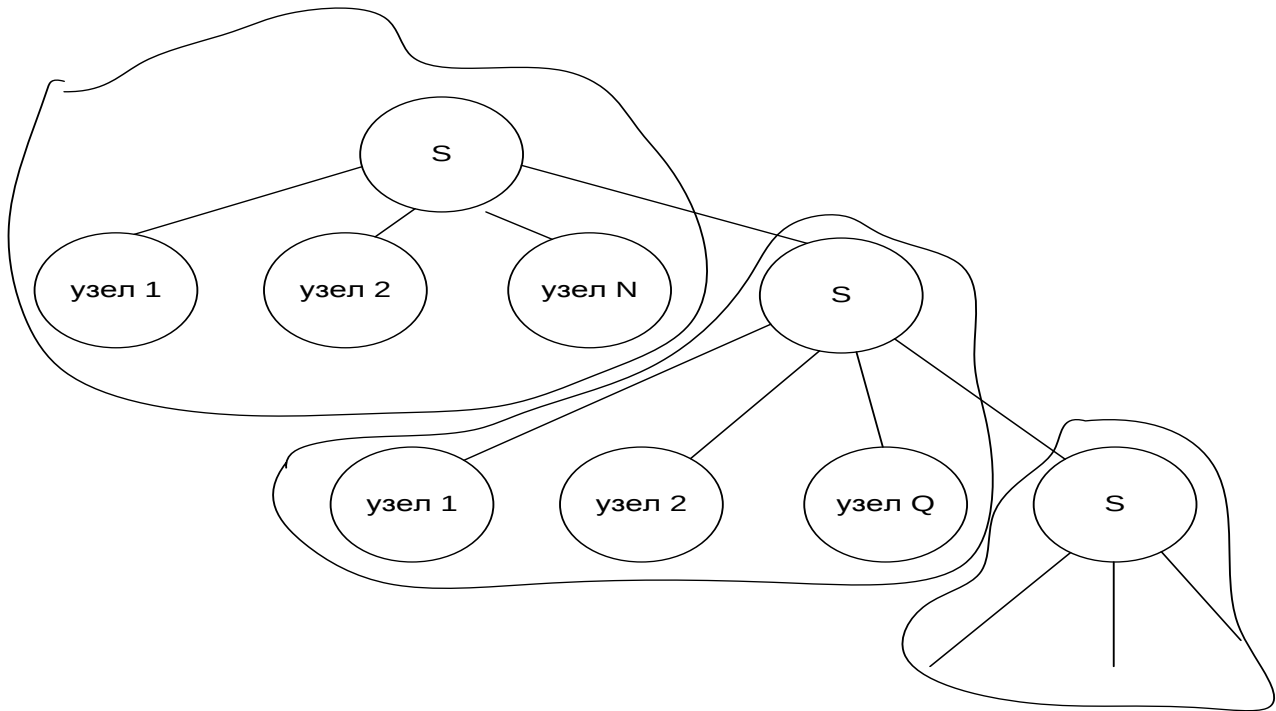
4) Дерево разбора

Дерево разбора:

S – стартовый символ.

узел 1, узел 2, ..., узел N – каждый узел описывает функцию.

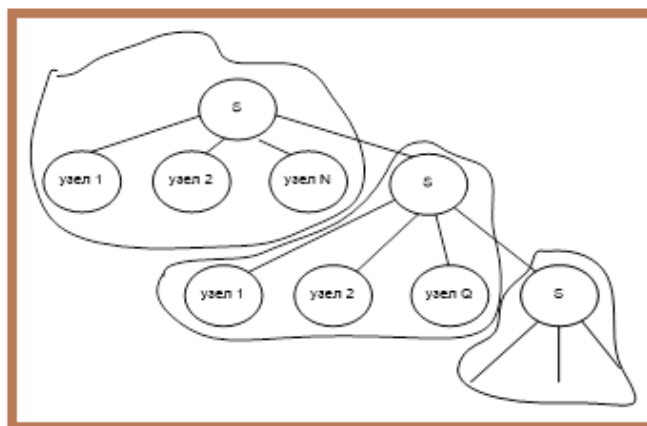
Каждой функции соответствует блок кода.



5) Модель памяти SVV-2015: точки входа функций и код

Точки входа функций

команда перехода в главную точку входа
команда перехода в блок кода 1
команда перехода в блок кода 2
команда перехода в блок кода 3
команда перехода в блок кода N

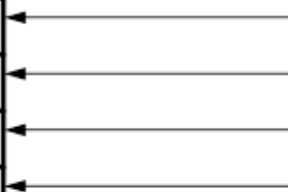


Код

главная точка входа
Пролог: выделение памяти, настройка адресов, согласование с ОС, старт главной функции
Блок кода 1 (главная функция)
Блок кода 2 (функция)
Блок кода 3 (функция)
Блок кода N (функция)
Эпилог: освобождение памяти, согласование с ОС

Дерево разбора

Поддерево 1
Поддерево 2
Поддерево 3
Поддерево N



6) Простой вариант генерации кода на ассемблере

Скелет главной функции:

```
.586                                ; система команд (процессор Pentium)
.model flat,stdcall                 ; модель памяти, соглашение о вызовах
includelib kernel32.lib            ; компоновщику: компоновать с kernel32.lib
                                   ; можем компоновать со стандартной библиотекой

ExitProcess PROTO :DWORD           ; прототип функции

.stack 4096                         ; сегмент стека объемом 4096 - для вычислений

.data                              ; сегмент данных - переменные и параметры

.const                             ; сегмент констант - литералы

.code                              ; сегмент кода - исполняемый код

main PROC                          ; начало процедуры - согласование с ОС

    push 0                         ; код возврата процесса (параметр ExitProcess )
    call ExitProcess               ; так должен заканчиваться любой процесс Windows
main ENDP                          ; конец процедуры
end main
```

7) Модель памяти SVV-2015: данные

```

; ----- данные функция fi -----
ret_fi  dword 0h      ; место для возврата значения функцией fi  ти:0

fix     dword 0h      ; параметр x функции fi                    ти:1

fiy     dword 0h      ; параметр y функции fi                    ти:2

fiz     dword 0h      ; переменная объявленная в функции fi    ти:3

;----- данные функции fs -----
ret_fs  byte  0h      ; место для возврата значения функцией fs ти:4
        byte 127 dup(0h)

fsa     byte  0h      ; параметр а функции fs                    ти:5
        byte 127 dup(0h)

fsb     byte  0h      ; параметр b функции fs                    ти:6
        byte 127 dup(0h)

fsc     byte  0h      ; переменная с функции fs                ти:7
        byte 127 dup(0h)

.const          ; сегмент констант - литералы

;----- данные функции fs -----
l001     dword 1h      ; литерал                                ти:9

l002     dword 3h      ; литерал                                ти:10

```

ти	0
имя	fi
тип	функция, int
иниц.	0x00000000

ти	1
имя	fix
тип	параметр, int
иниц.	0x00000000

ти	2
имя	fiy
тип	параметр, int
иниц.	0x00000000

ти	3
имя	fiz
тип	переменная, int
иниц.	0x00000000

ти	4
имя	fs
тип	функция, string
иниц.	0x00

ти	5
имя	fsa
тип	параметр, string
иниц.	0x00

ти	6
имя	fsb
тип	параметр, string
иниц.	0x00

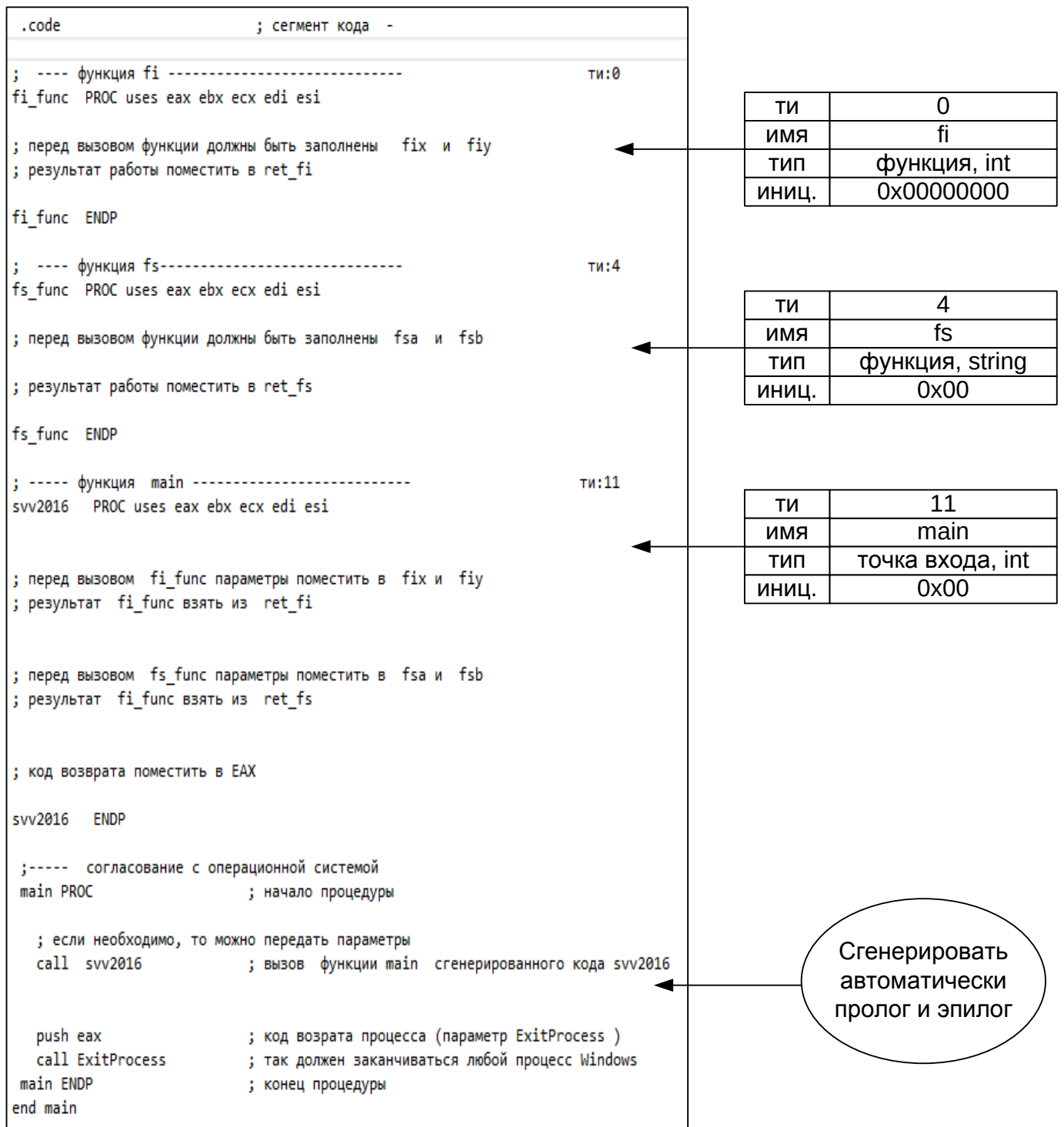
ти	7
имя	fsc
тип	переменная, string
иниц.	0x00

ти	8
имя	substr
тип	вн.функц, string
иниц.	0x00

ти	9
имя	l001
тип	литерал, int
иниц.	0x00000001

ти	10
имя	l002
тип	литерал, int
иниц.	0x00000003

8) Заготовки-шаблоны для функций



Для внешних функций можно сгенерировать функцию-обертку для согласования стандарта вызова:

- функцию написать на C++ и поместить ее библиотеку;
- вызов функции выполнить из функции обертки;
- в генерируемом коде вызывать функцию обертку.

9) Генерация кода для выражений

```
.data                                ; сегмент данных - переменные и параметры
svv2016x    sdword oh                ; переменная x со знаком
svv2016y    sdword oh                ; переменная y со знаком
svv2016z    sdword oh                ; переменная z со знаком
svv2016a    sdword oh                ; переменная a со знаком
svv2016b    sdword oh                ; переменная b со знаком

.const                                           ; сегмент констант - литералы
1001    sdword 1h                      ; литерал
1002    sdword 3h                      ; литерал
1003    sdword 4h                      ; литерал

.code                                           ; сегмент кода - исполняемый код
```

```
;-----
; x = 1;
; y = x;
; z = x*y;
; z = z + x*y;
; a = 3;
; b = 7;
; z = x*y + b*(x+y);
```

```
; генерация кода: x = 1 --> svv2016x = 1001
```

```
    ; генерация кода: 1001
    push 1001
```

```
    ; генерация кода: x=
    pop  svv2016x
```

```
; генерация кода: y = x --> svv2016y = svv2016x
```

```
    ; генерация кода: svv2016x
    push svv2016x
```

```
    ; генерация кода: y=
    pop  svv2016y
```

```

; генерация кода: z = x*y  --> svv2016z = svv2016x*svv2016y -->svv2016z = svv2016x svv2016y *

; генерация кода: svv2016x
push svv2016x

; генерация кода: svv2016y
push svv2016y

; генерация кода: *
pop eax
pop ebx
imul ebx ; eax = eax*ebx
push eax

; генерация кода: z =
pop svv2016z

```

```

; генерация кода: z = z+ x*y  --> svv2016z = svv2016z+ svv2016x*svv2016y -->svv2016z = svv2016z svv2016x svv2016y *+

; генерация кода: svv2016z
push svv2016z

; генерация кода: svv2016x
push svv2016x

; генерация кода: svv2016y
push svv2016y

; генерация кода: *
pop eax
pop ebx
imul ebx ; eax = eax*ebx
push eax

; генерация кода: +
pop eax
pop ebx
add eax, ebx ; eax = eax+ebx
push eax

; генерация кода: z =
pop svv2016z

```

```

; a = 3 --> a = 1002
; b = 7 ---> b = 1003
; z = x*y + b*(x+y) --> svv2016z = svv2016x*svv2016y + svv2016b * (svv2016x + svv2016y)
;                               svv2016z = svv2016x svv2016y * svv2016b svv2016x svv2016y ++
; генерация кода: a = 3 --> svv2016a = 1002

; генерация кода: 1002
    push 1002
; генерация кода: a =
    pop svv2016a
; генерация кода: 1003
    push 1003
; генерация кода: b =
    pop svv2016b
; генерация кода: svv2016x
    push svv2016x
; генерация кода: svv2016y
    push svv2016y
; генерация кода: *
    pop    eax
    pop    ebx
    imul   ebx        ; eax = eax*ebx
    push   eax
; генерация кода: svv2016b
    push svv2016b
; генерация кода: svv2016x
    push svv2016x
; генерация кода: svv2016y
    push svv2016y
; генерация кода: +
    pop    eax
    pop    ebx
    add    eax, ebx    ; eax = eax+ebx
    push   eax
; генерация кода: *
    pop    eax
    pop    ebx
    imul   ebx        ; eax = eax*ebx
    push   eax
; генерация кода: +
    pop    eax
    pop    ebx
    add    eax, ebx    ; eax = eax+ebx
    push   eax
; генерация кода: svv2016z =
    pop svv2016z

```

10) Готовые шаблоны:

```
//шаблоны
#define EXPR_INT      "push    %s \n"           // i
#define EXPR_INT_E    "pop     %s \n"           // =
#define EXPR_INT_PLUS "pop     eax\npop     ebx\nadd    eax, ebx\npush  eax\n"           // +
#define EXPR_INT_MUL  "pop     eax\npop     ebx\nimul   eax, ebx\npush  eax\n"           // *
#define GEN1(b, tmpl, var) sprintf_s(b, 1024,tmpl, #var)
#define GEN0(b, tmpl)   sprintf_s(b, 1024,tmpl)
```

Пояснения:

- *<имя>_s* – это безопасные функции с указанием емкости приемника, например:
sprintf_s – возвращает количество байт, записанных в буфер или -1
- *Буфер* – место, где сохраняется генерируемый код (**b**).
- *Стрингификация* – операция # (преобразование фрагмента кода в строку)

```
char buf[1024];

// x =1
int k = GEN1(buf,EXPR_INT,      1001);           // 1
      k+= GEN1(buf+k,EXPR_INT_E,  svv2016x);      // =
```

```
// y = x
      k+= GEN1(buf+k,EXPR_INT,      svv2016x);    // x
      k+= GEN1(buf+k,EXPR_INT_E,    svv2016y);    // =
```

```
// z = xy*
      k+= GEN1(buf+k,EXPR_INT,      svv2016x);    // x
      k+= GEN1(buf+k,EXPR_INT,      svv2016y);    // y
      k+= GEN0(buf+k,EXPR_INT_MUL);  // *
      k+= GEN1(buf+k,EXPR_INT_E,    svv2016z);    // =
```

```
// z = zxy*+
      k+= GEN1(buf+k,EXPR_INT,      svv2016z);    // z
      k+= GEN1(buf+k,EXPR_INT,      svv2016x);    // x
      k+= GEN1(buf+k,EXPR_INT,      svv2016y);    // y
      k+= GEN0(buf+k,EXPR_INT_MUL);  // *
      k+= GEN0(buf+k,EXPR_INT_PLUS); // +
      k+= GEN1(buf+k,EXPR_INT_E,    svv2016z);    // =
```

```
// a = 3;
k+= GEN1(buf+k,EXPR_INT,      1002);           // 3
k+= GEN1(buf+k,EXPR_INT_E,    svv2016a);       // =
```

```
// b = 7;
k+= GEN1(buf+k,EXPR_INT,      1003);           // 7
k+= GEN1(buf+k,EXPR_INT_E,    svv2016b);       // =
```

```
// z = xy* bxy*++;
k+= GEN1(buf+k,EXPR_INT,      svv2016x);       // x
k+= GEN1(buf+k,EXPR_INT,      svv2016y);       // y
k+= GEN0(buf+k,EXPR_INT_MUL);                  // *
k+= GEN1(buf+k,EXPR_INT,      svv2016b);       // b
k+= GEN1(buf+k,EXPR_INT,      svv2016x);       // x
k+= GEN1(buf+k,EXPR_INT,      svv2016y);       // y
k+= GEN0(buf+k,EXPR_INT_PLUS);                  // +
k+= GEN0(buf+k,EXPR_INT_MUL);                  // *
k+= GEN0(buf+k,EXPR_INT_PLUS);                  // +
k+= GEN1(buf+k,EXPR_INT_E,    svv2016z);       // =

std::cout << buf;
```

11) Сгенерированный код

```
push    1001
pop     svv2016x
push    svv2016x
pop     svv2016y
push    svv2016x
push    svv2016y
pop     eax
pop     ebx
imul    eax, ebx
push    eax
pop     svv2016z
push    svv2016z
push    svv2016x
push    svv2016y
pop     eax
pop     ebx
imul    eax, ebx
push    eax
pop     eax
pop     ebx
add     eax, ebx
push    eax
pop     svv2016z
push    1002
pop     svv2016a
push    1003
pop     svv2016b
push    svv2016x
push    svv2016y
pop     eax
pop     ebx
imul    eax, ebx
push    eax
push    svv2016b
push    svv2016x
push    svv2016y
pop     eax
pop     ebx
add     eax, ebx
push    eax
pop     eax
pop     ebx
imul    eax, ebx
push    eax
pop     eax
pop     ebx
add     eax, ebx
push    eax
pop     svv2016z
```

Последовательность генерации:

- 1) План памяти строится по ТИ
- 2) Выполняется преобразование выражений в ПОЛИЗ
- 3) Заготавливаются шаблоны кода соответствующего правила грамматики
- 4) Код генерируется по дереву разбора